ЛАБОРАТОРНА РОБОТА № 1

Тема: Робота з базовими типами даних

Мета: Ознайомитись з основними можливостями мови програмування Go, найпростішої структурою програми, з властивостями базових типів. Отримати практичні навички по роботі з базовими типами даних в мові Go.

хід роботи

Завдання 1.

1. Вивести текст українською мовою

Виконання завдання:

```
func main() {
   // Оголошення змінної
   var str string = "Студент групи ВТк-24-1"
   //Завдання.
   // 1. Вивести текст українською мовою
   fmt.Println("Привіт, мене звати Роман! ", str)
}
```

Результат виконання:

```
Привіт, мене звати Роман!  Студент групи ВТк-24-1

Process finished with the exit code 0
```

Рис.1.1

Завдання 2.

1. Створити цілочисельну змінну (результат не відображати)

Виконання завдання:

```
func main() {
    // Усі змінні ініціалізуються значенням за замовчуванням, тобто 0

    var defaultInt8 int8
    var defaultInt16 int16
    var defaultInt32 int32
    var defaultInt64 int64
    var defaultInt int

    fmt.Println("Default values (signed): ", defaultInt8, defaultInt16, defaultInt32,
defaultInt64, defaultInt)

    var defaultuInt8 uint8
    var defaultuInt16 uint16
    var defaultuInt32 uint32
    var defaultuInt64 uint64
    var defaultuInt uint

    fmt.Println("Default values (unsigned): ", defaultuInt8, defaultuInt16, defaultuInt32,
defaultuInt64, defaultuInt)

    // Завдання.
    //1. Створити цілочисельну змінну (результат не відображати)
    var taskInt int32
    _ = taskInt // використовуємо змінну, щоб уникнути помилки компіляції
}
```

Результат виконання:

```
Default values (signed):  0 0 0 0 0
Default values (unsigned):  0 0 0 0 0

Process finished with the exit code 0
```

Рис. 1.2

Завдання 3.

1. Вивести типи всіх змінних

2. Присвоїти змінній intVar змінні userinit16 і userautoinit. Результат вивести в консоль.

Виконання завдання:

```
func main() {
   // Ініціалізація змінних
   var userinit8 uint8 = 1
   var userinit16 uint16 = 2
   var userinit64 int64 = -3
   var userautoinit = -4 // Такий варіант ініціалізації також можливий

   fmt.Println("Values: ", userinit8, userinit16, userinit64, userautoinit, "\n")

   // Короткий запис оголошення змінної
   // тільки для нових змінних
   intVar := 10

   fmt.Printf("Value = %d Type = %T\n", intVar, intVar)

   // Завдання.
   // 1. Вивести типи всіх змінних
   fmt.Printf("userinit8 Type = %T\n", userinit8)
   fmt.Printf("userinit16 Type = %T\n", userinit16)
   fmt.Printf("userinit64 Type = %T\n", userinit64)
   fmt.Printf("userautoinit Type = %T\n", userautoinit)
   fmt.Printf("intVar Type = %T\n\n", intVar)

   // 2. Присвоїти змінній intVar змінні userinit16 і userautoinit. Результат вивести в
консоль.
   intVar = int(userinit16)
   fmt.Printf("intVar = userinit16: Value = %d\n", intVar)
   intVar = int(userautoinit)
   fmt.Printf("intVar = userautoinit: Value = %d\n", intVar)
}
```

Результат виконання:

```
Values:  1 2 -3 -4

Value = 10 Type = int
userinit8 Type = uint8
userinit16 Type = uint16
userinit64 Type = int64
userautoinit Type = int
intVar Type = int
```

Рис. 1.3 – Результат завдання 3.1

```
intVar = userinit16: Value = 2
intVar = userautoinit: Value = -4
```

Рис. 1.4 – Результат завдання 3.2

Завдання 4.

1. Виконати вираз: fmt.Println("c-- = ", c--)

Виконання завдання:

```
func main() {
   // Ініціалізація змінних
   var a int16 = 2
   var b int16 = 3
   var c int64 = 10

   fmt.Println("a + b       = ", a+b)
   fmt.Println("a - b       = ", a-b)
   fmt.Println("a * b       = ", a*b)
   fmt.Println("int(a / b)   = ", int(a/b), "\n")

   c--
   fmt.Println("c--     = ", c)
   c++
   fmt.Println("c++     = ", c)
   c += 10
   fmt.Println("c += 10 = ", c)
   c -= 5
   fmt.Println("c -= 5  = ", c)
   c *= 3
   fmt.Println("c *= 3  = ", c)
   c /= 7
   fmt.Println("c /= 7  = ", c)

   // Завдання.
   // 1. Виконати вираз: fmt.Println("c--     = ", c--)
```

```
    // Примітка: В Go не можна використовувати інкремент/декремент в виразах, тільки як окремі оператори
    c--
    fmt.Println("c--     = ", c)
}
```

Результат виконання:

```
c--     =  9
```

Рис. 1.5

Завдання 5.

1. Визначити розрядність платформи

Виконання завдання:

```
func main() {
    fmt.Println("Синоніми цілих типів\n")

    fmt.Println("byte    - int8")
    fmt.Println("rune    - int32")
    fmt.Println("int     - int32 або int64, в залежності від платформи")
    fmt.Println("uint    - uint32 або uint64, в залежності від платформи")

    // Завдання.
    // 1. Визначити розрядність платформи
    fmt.Printf("\nРозрядність int: %d біт\n", 32<<(^uint(0)>>63))
    fmt.Printf("Розрядність uint: %d біт\n", 32<<(^uint(0)>>63))
}
```

Результат виконання:

```
byte    - int8
rune    - int32
int     - int32 або int64, в залежності від платформи
uint    - uint32 або uint64, в залежності від платформи

Розрядність int: 64 біт
Розрядність uint: 64 біт
```

Рис. 1.6

Завдання 6.

1. Пояснити результати операцій

Виконання завдання:

```
func main() {
    var x, y, z uint8

    x = 9
    y = 28
```

```
    z = x

    fmt.Println("Бітові операції")

    fmt.Printf("^x      = ^(%d)      = ^(%.8b)            = %.8b = %d\n", x, x, ^x, ^x)
    fmt.Printf("x << 2  = (%d << 2)  = (%.8b << 2)        = %.8b = %d\n", x, x, x<<2,
x<<2)
    fmt.Printf("x >> 2  = (%d >> 2)  = (%.8b >> 2)        = %.8b = %d\n", x, x, x>>2,
x>>2)
    fmt.Printf("x & y   = (%d & %d)  = (%.8b & %.8b)  = %.8b = %d\n", x, y, x, y, x&y,
x&y)
    fmt.Printf("x | y   = (%d | %d)  = (%.8b | %.8b)  = %.8b = %d\n", x, y, x, y, x|y, x|y)
    fmt.Printf("x ^ y   = (%d ^ %d)  = (%.8b ^ %.8b)  = %.8b = %d\n", x, y, x, y, x^y,
x^y)
    fmt.Printf("x &^ y  = (%d &^ %d) = (%.8b &^ %.8b) = %.8b = %d\n", x, y, x, y,
x&^y, x&^y)
    fmt.Printf("x %% y   = (%d %% %d)  = (%.8b %% %.8b)  = %.8b = %d\n", x, y, x, y,
x%y, x%y)

    fmt.Println("\nБітові операції з присвоюванням")

    x = z
    x &= y
    fmt.Printf("x &= y   = (%d &= %d)  = (%.8b &= %.8b)  = %.8b = %d\n", z, y, z, y, x,
x)
    x = z
    x |= y
    fmt.Printf("x |= y   = (%d |= %d)  = (%.8b |= %.8b)  = %.8b = %d\n", z, y, z, y, x, x)
    x = z
    x ^= y
    fmt.Printf("x ^= y   = (%d ^= %d)  = (%.8b ^= %.8b)  = %.8b = %d\n", z, y, z, y, x,
x)
    x = z
    x &^= y
    fmt.Printf("x &^= y  = (%d &^= %d) = (%.8b &^= %.8b) = %.8b = %d\n", z, y, z, y,
x, x)
    x = z
    x %= y
    fmt.Printf("x %%= y   = (%d %%= %d)  = (%.8b %%= %.8b)  = %.8b = %d\n", z, y,
z, y, x, x)

    // Завдання.
    // 1. Пояснити результати операцій
```

```
    fmt.Println("\nПояснення:")

    fmt.Println("^x: інверсія всіх бітів числа x (00001001 -> 11110110 = 246)")
    fmt.Println("x << 2: зсув вліво на 2 біти (множення на 4) = 36")
    fmt.Println("x >> 2: зсув вправо на 2 біти (ділення на 4) = 2")

    fmt.Println("x & y: AND — 1 тільки якщо обидва біти = 1 (результат 8)")
    fmt.Println("x | y: OR — 1 якщо хоча б один біт = 1 (результат 29)")
    fmt.Println("x ^ y: XOR — 1 якщо біти різні (результат 21)")
    fmt.Println("x &^ y: AND NOT — очищає біти x, де в y є 1 (результат 1)")

    fmt.Println("x % y: остача від ділення (9 % 28 = 9)")

    fmt.Println("\nОперації з присвоюванням:")
    fmt.Println("x &= y → x = x & y = 8")
    fmt.Println("x |= y → x = x | y = 29")
    fmt.Println("x ^= y → x = x ^ y = 21")
    fmt.Println("x &^= y → x = x &^ y = 1")
    fmt.Println("x %= y → x = x % y = 9")
}
```

Результат виконання:

```
Пояснення:
^x: інверсія всіх бітів числа x (00001001 -> 11110110 = 246)
x << 2: зсув вліво на 2 біти (множення на 4) = 36
x >> 2: зсув вправо на 2 біти (ділення на 4) = 2
x & y: AND — 1 тільки якщо обидва біти = 1 (результат 8)
x | y: OR — 1 якщо хоча б один біт = 1 (результат 29)
x ^ y: XOR — 1 якщо біти різні (результат 21)
x &^ y: AND NOT — очищає біти x, де в y є 1 (результат 1)
x % y: остача від ділення (9 % 28 = 9)

Операції з присвоюванням:
x &= y → x = x & y = 8
x |= y → x = x | y = 29
x ^= y → x = x ^ y = 21
x &^= y → x = x &^ y = 1
x %= y → x = x % y = 9

Process finished with the exit code 0
```

Рис. 1.7

Завдання 7.

1. Створіть 2 змінні різних типів. Виконайте арифметичні операції. Результат вивести в консоль

Виконання завдання:

```
func main() {
    variable8 := int8(127)
```

```
    variable16 := int16(16383)

    fmt.Println("Приведення типів\n")

    fmt.Printf("variable8         = %-5d = %.16b\n", variable8, variable8)
    fmt.Printf("variable16        = %-5d = %.16b\n", variable16, variable16)
    fmt.Printf("uint16(variable8) = %-5d = %.16b\n", uint16(variable8),
uint16(variable8))
    fmt.Printf("uint8(variable16) = %-5d = %.16b\n", uint8(variable16),
uint8(variable16))

    // Завдання.
    // 1. Створіть 2 змінні різних типів. Виконайте арифметичні операції. Результат
вивести в консоль
    var num1 int32 = 15
    var num2 float64 = 3.5

    fmt.Println("\nАрифметичні операції з різними типами:")
    fmt.Printf("num1 = %d (тип: %T)\n", num1, num1)
    fmt.Printf("num2 = %.2f (тип: %T)\n\n", num2, num2)

    // Приведення типів для арифметичних операцій
    fmt.Printf("float64(num1) + num2 = %.2f\n", float64(num1)+num2)
    fmt.Printf("float64(num1) - num2 = %.2f\n", float64(num1)-num2)
    fmt.Printf("float64(num1) * num2 = %.2f\n", float64(num1)*num2)
    fmt.Printf("float64(num1) / num2 = %.2f\n", float64(num1)/num2)
}
```

Результат виконання:

```
Приведення типів

variable8         = 127   = 0000000001111111
variable16        = 16383 = 0011111111111111
uint16(variable8) = 127   = 0000000001111111
uint8(variable16) = 255   = 0000000011111111

Арифметичні операції з різними типами:
num1 = 15 (тип: int32)
num2 = 3.50 (тип: float64)

float64(num1) + num2 = 18.50
float64(num1) - num2 = 11.50
float64(num1) * num2 = 52.50
float64(num1) / num2 = 4.29

Process finished with the exit code 0
```

Рис. 1.8

Завдання 8.

1. Створіть змінні різних типів, використовуючи короткий запис та ініціалізацію за замовчуванням. Результат вивести в консоль

Виконання завдання:

```
func main() {
    var defaultFloat float32
    var defaultDouble float64 = 5.5

    fmt.Println("defaultfloat      = ", defaultFloat)
    fmt.Printf("defaultDouble (%T) = %f\n\n", defaultDouble, defaultDouble)

    fmt.Println("MAX float32       = ", math.MaxFloat32)
    fmt.Println("MIN float32       = ", math.SmallestNonzeroFloat32, "\n")

    fmt.Println("MAX float64       = ", math.MaxFloat64)
    fmt.Println("MIN float64       = ", math.SmallestNonzeroFloat64, "\n")

    // Завдання.
    // 1. Створіть змінні різних типів, використовуючи короткий запис та
ініціалізацію за замовчуванням. Результат вивести в консоль
    var defaultInt int
    var defaultFloat2 float32
    var defaultBool bool
    var defaultString string

    shortInt := 42
    shortFloat := 3.14
    shortBool := true
```

```
    shortString := "Go"

    fmt.Println("\nЗмінні з ініціалізацією за замовчуванням:")
    fmt.Printf("defaultInt = %d (тип: %T)\n", defaultInt, defaultInt)
    fmt.Printf("defaultFloat2 = %f (тип: %T)\n", defaultFloat2, defaultFloat2)
    fmt.Printf("defaultBool = %v (тип: %T)\n", defaultBool, defaultBool)
    fmt.Printf("defaultString = \"%s\" (тип: %T)\n\n", defaultString, defaultString)

    fmt.Println("Змінні з коротким записом:")
    fmt.Printf("shortInt = %d (тип: %T)\n", shortInt, shortInt)
    fmt.Printf("shortFloat = %.2f (тип: %T)\n", shortFloat, shortFloat)
    fmt.Printf("shortBool = %v (тип: %T)\n", shortBool, shortBool)
    fmt.Printf("shortString = \"%s\" (тип: %T)\n", shortString, shortString)
}
```

Результат виконання:

```
Змінні з ініціалізацією за замовчуванням:
defaultInt = 0 (тип: int)
defaultFloat2 = 0.000000 (тип: float32)
defaultBool = false (тип: bool)
defaultString = "" (тип: string)

Змінні з коротким записом:
shortInt = 42 (тип: int)
shortFloat = 3.14 (тип: float64)
shortBool = true (тип: bool)
shortString = "Go" (тип: string)
```

Рис. 1.9

Завдання 9.

1. Пояснити результати операцій

Виконання завдання:

```
func main() {
    var first, second bool
    var third bool = true
    fourth := !third
    var fifth = true

    fmt.Println("first  = ", first)       // false
    fmt.Println("second = ", second)      // false
    fmt.Println("third  = ", third)       // true
    fmt.Println("fourth = ", fourth)      // false
    fmt.Println("fifth  = ", fifth, "\n") // true

    fmt.Println("!true  = ", !true)       // false
    fmt.Println("!false = ", !false, "\n") // true
```

```
    fmt.Println("true && true   = ", true && true)        // true
    fmt.Println("true && false  = ", true && false)       // false
    fmt.Println("false && false = ", false && false, "\n") // false

    fmt.Println("true || true   = ", true || true)        // true
    fmt.Println("true || false  = ", true || false)       // true
    fmt.Println("false || false = ", false || false, "\n") // false

    fmt.Println("2 < 3  = ", 2 < 3)       // true
    fmt.Println("2 > 3  = ", 2 > 3)       // false
    fmt.Println("3 < 3  = ", 3 < 3)       // false
    fmt.Println("3 <= 3 = ", 3 <= 3)      // true
    fmt.Println("3 > 3  = ", 3 > 3)       // false
    fmt.Println("3 >= 3 = ", 3 >= 3)      // true
    fmt.Println("2 == 3 = ", 2 == 3)      // false
    fmt.Println("3 == 3 = ", 3 == 3)      // true
    fmt.Println("2 != 3 = ", 2 != 3)      // true
    fmt.Println("3 != 3 = ", 3 != 3, "\n") // false

    // Завдання.
    // 1. Пояснити результати операцій
    fmt.Println("\nПояснення результатів операцій:")
    fmt.Println("! - логічне НІ: інвертує значення (true -> false, false -> true)")
    fmt.Println("&& - логічне І: повертає true тільки якщо обидва операнди true")
    fmt.Println("|| - логічне АБО: повертає true якщо хоча б один операнд true")
    fmt.Println("<, >, <=, >= - операції порівняння: порівнюють числові значення")
    fmt.Println("== - рівність: перевіряє чи рівні значення")
    fmt.Println("!= - нерівність: перевіряє чи не рівні значення")
}
```

Результат виконання:

```
Пояснення результатів операцій:
! - логічне НІ: інвертує значення (true -> false, false -> true)
&& - логічне І: повертає true тільки якщо обидва операнди true
|| - логічне АБО: повертає true якщо хоча б один операнд true
<, >, <=, >= - операції порівняння: порівнюють числові значення
== - рівність: перевіряє чи рівні значення
!= - нерівність: перевіряє чи не рівні значення
```

Рис. 1.10

Завдання 10.

1. Вивести українську літеру 'Ї'
2. Пояснити призначення типу "rune"

Виконання завдання:

```
func main() {
   var chartype int8 = 'R'

   fmt.Printf("Code '%c' - %d\n", chartype, chartype)

   // Завдання.
   // 1. Вивести українську літеру 'Ї'
   var ukrainianLetter rune = 'Ї'
   fmt.Printf("Code '%c' - %d\n", ukrainianLetter, ukrainianLetter)

   // 2. Пояснити призначення типу "rune"
   fmt.Println("\nПояснення типу 'rune':")
   fmt.Println("Тип 'rune' є синонімом типу int32 і використовується для представлення")
   fmt.Println("кодових точок Unicode. Він дозволяє коректно працювати з символами")
   fmt.Println("різних мов, включаючи українські літери, які потребують більше 1 байта.")
   fmt.Println("rune забезпечує коректну роботу з багатобайтовими символами UTF-8.")
}
```

Результат виконання:

```
Code 'Ї' - 1031
```

Рис. 1.11 – Результат завдання 10.1

```
Пояснення типу 'rune':
Тип 'rune' є синонімом типу int32 і використовується для представлення
кодових точок Unicode. Він дозволяє коректно працювати з символами
різних мов, включаючи українські літери, які потребують більше 1 байта.
rune забезпечує коректну роботу з багатобайтовими символами UTF-8.
```

Рис. 1.12 – Результат завдання 10.2

Висновок

У ході виконання завдання було розглянуто бітові операції в мові Go.

Було встановлено, що побітові операції виконуються над кожним бітом окремо, а також можуть використовуватись у поєднанні з операторами присвоювання. У ході виконання завдання було розглянуто бітові операції в мові Go.

Було встановлено, що побітові операції виконуються над кожним бітом окремо, а також можуть використовуватись у поєднанні з операторами присвоювання.

Контрольні питання

1. Перерахуйте типи даних та поясніть їх основні властивості.

У мові Go існують такі основні типи даних:

- **Цілочисельні (int, int8, int16, int32, int64)**
  Використовуються для зберігання цілих чисел.
- **Дійсні (float32, float64)**
  Використовуються для чисел з плаваючою комою.
- **Логічний тип (bool)**
  Має два значення: true або false.
- **Рядки (string)**
  Використовуються для зберігання тексту.
- **Комплексні числа (complex64, complex128)**
  Для роботи з комплексними числами.

Основні властивості:

- кожен тип має фіксований розмір у пам'яті
- тип визначає допустимі операції
- строга типізація (не можна змішувати типи без приведення)

2. Яка структура програми на мові Go?
Програма на Go має таку структуру:

```
package main

import "fmt"

func main() {
   // тіло програми
}
```

Пояснення:

- package main — головний пакет
- import — підключення бібліотек
- func main() — точка входу в програму

3. Які існують цілочисельні типи даних?
У Go є такі цілочисельні типи:

Знакові:

- int8, int16, int32, int64
- int (залежить від платформи)

Беззнакові:

- uint8, uint16, uint32, uint64
- uint
- byte (аналог uint8)

4. Яка різниця між цілочисельні типами даних?
Різниця полягає у:

Розмірі (кількість біт):

- int8 → 8 біт
- int16 → 16 біт
- і т.д.

Діапазоні значень:

- int8 → від -128 до 127
- uint8 → від 0 до 255

Наявності знаку:

- int → може бути від'ємним
- uint → тільки додатні числа

5. Як зберігаються строки в мові Go?
У Go строки:

- зберігаються як послідовність байтів (UTF-8)
- є незмінними (immutable)
- беруться в подвійні лапки " "

Приклад:
str := "Hello"
Особливості:

- не можна змінити окремий символ напряму
- можна отримати довжину через len()
- підтримують Unicode